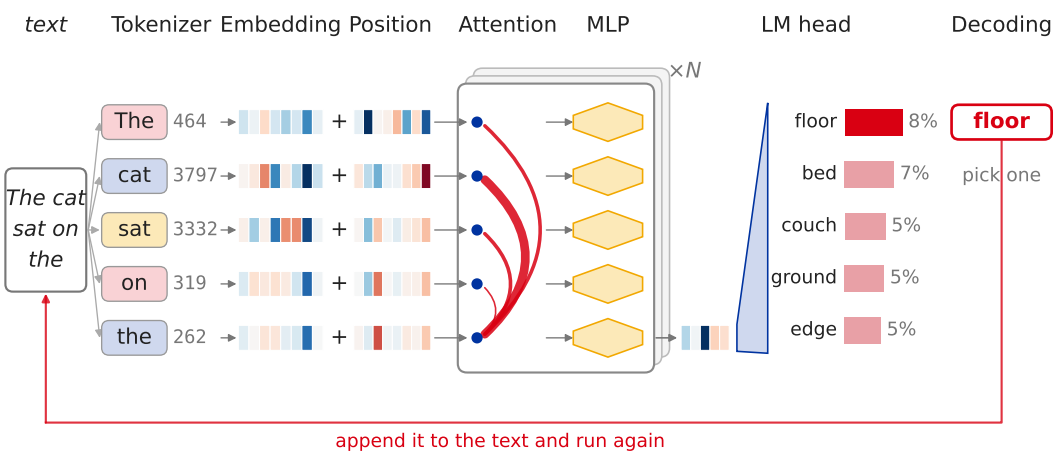
Fine-tuning on a Budget

LoRA and QLoRA

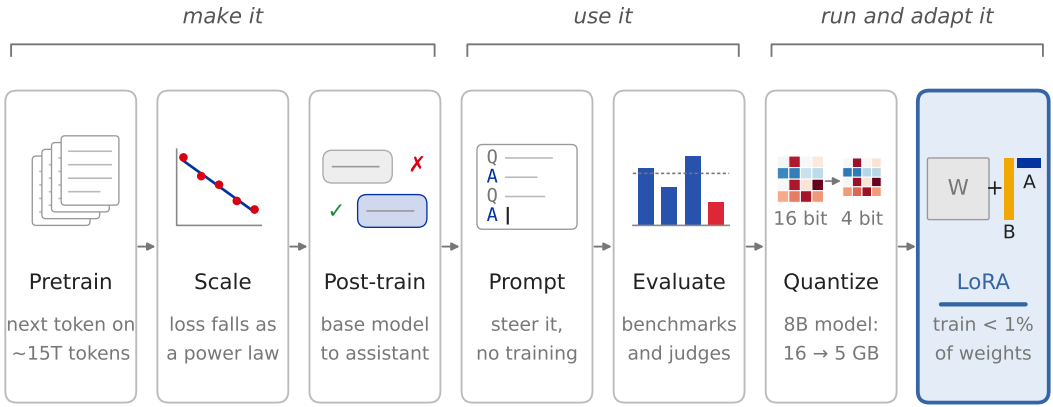
LLM chapter, session 13. Toy numbers unless a slide says otherwise.

Where we are: one forward pass



The same pass. Today we change a few of its weights - cheaply.

Where we are: the life of a model



Today: the last box. Adapting a model to your task on hardware you can get.

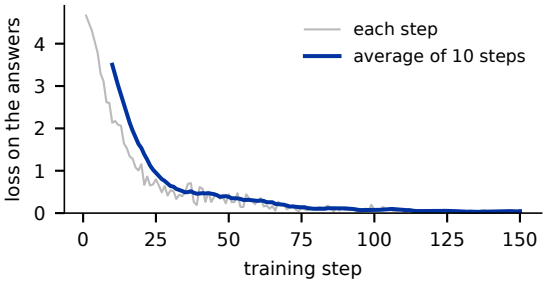
How much do you need to change?

Full fine-tuning updates every weight. For a 7B model that needs about 112 GB of memory (LLM-8's 16 bytes per parameter) - several big GPUs.

We want GPT-2 to open every answer with "Good question!". Out of the box, it never does.

Predict first: what share of GPT-2's 124M weights must we train to teach it that habit - 50%? 10%? 1%?

0.24% was enough



Real run on this laptop, cached GPT-2, written by hand (no library): 294,912 new weights (0.24%) trained for 150 steps on 32 short conversations; everything else frozen. A few minutes of CPU.

20 questions it never saw, answers opening with "Good question!": **0 of 20** before, **20 of 20** after. (Closest guess: 1%; the count is derived in a few pages.)

"What is the capital of Italy?" $\rightarrow$ "Good question! Rome." But "How many wheels does a bicycle have?" $\rightarrow$ "Good question! Eight." - wrong: the habit transferred, the facts did not improve.

That is LoRA. Today: how a quarter of a percent can be enough, and how to do it to a 7B model on a free GPU.

Outline

Should you fine-tune?

LoRA

QLoRA

The recipe

Should you fine-tune?

Usually not first.

The ladder

1. **Prompt** (LLM-10): free, instant, reversible. Try it first.
2. **Give it documents** (RAG): for facts it does not know or that change.
3. **Fine-tune**: for a format, a style, a narrow task done thousands of times.

A fourth option for a whole new domain or language: **continued pretraining** - LLM-7's objective on more text, for example Armenian news (as arm-gemma did, LLM-11).

Fine-tuning teaches *how* to answer far better than *what* is true (LIMA, LLM-9). Our demo changed GPT-2's habit, not its knowledge: asked for the capital of Armenia, it now answers politely - "Good question! Ararat." (It is Yerevan.)

LoRA

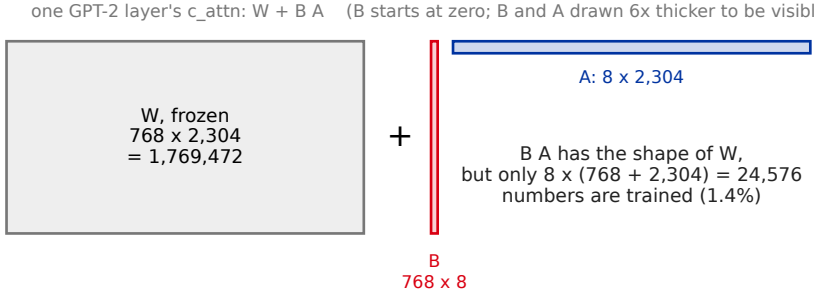
Train a small correction, not the whole matrix.

The bill for full fine-tuning

- Every trained weight needs its gradient and Adam's two running averages, plus a 32-bit copy: 16 bytes per parameter (LLM-8).
- A 7B model: $7 \times 10^9 \times 16 = 112$ GB, before activations.
- And every fine-tuned version is a full new copy of the model.

The memory goes to the parameters you train. Train fewer, and most of the bill disappears.

W + BA



- Keep W frozen. Learn its *change* as a product of two thin matrices, $\Delta W = BA$: a matrix of **rank** 8 at most (every row a mix of 8 directions). GPT-2's layer computes xW, so with the adapter (the pair B, A): $xW + \frac{\alpha}{r} xBA$.
- A starts random, B at zero: $BA = 0$, so step 0 is *exactly* the base model (our run: largest logit difference 0). Not both zero - then neither would get a gradient.
- α/r scales the update; ours, $16/8 = 2$. Change r and α keeps the update's size, so the learning rate need not be retuned.

The count, by hand

One GPT-2 layer's attention matrix `c_attn`, $768 \times 2{,}304$:

the matrix itself	$768 \times 2{,}304 = 1{,}769{,}472$
a rank-8 LoRA: B and A	$8 \times (768 + 2{,}304) = 24{,}576 \quad (1.4\%)$
the same in all 12 layers	$12 \times 24{,}576 = 294{,}912$
share of GPT-2's 124,439,808	**0.24%**

Only these are trained, so only these need gradients and Adam's state. The 16 bytes per parameter now apply to 0.3M weights, not 124M.

Why a low rank is enough

- A fine-tune does not rebuild the model; it nudges it. LoRA's hypothesis: that nudge, ΔW, has a low "intrinsic rank".
- Hu et al. (2021) tested it on GPT-3: adapting the attention matrices, accuracy on their tasks was already good at rank 1 and barely changed above it.
- On GPT-3 175B, LoRA cut the trainable parameters 10,000$\times$ and GPU memory 3$\times$, with quality on par with full fine-tuning.

"On par" was measured on tasks like the ones in the paper. A task far from what the base model knows may need a higher rank, more matrices - or full fine-tuning.

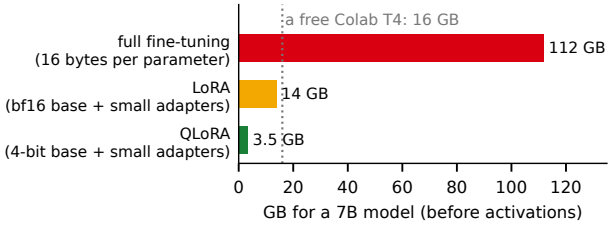
Which matrices, and after training

- The original LoRA adapted attention only. QLoRA (2023) found it needed adapters on **every** linear layer, attention and MLP, to match full fine-tuning. Our demo used `c_attn` only - enough for a habit.
- **Merge**: $W' = W + \frac{\alpha}{r} BA$ is a normal matrix again, so the merged model is as fast as the original. In our run, merging changed no logit by more than 0.0002 (rounding). With a 4-bit base (QLoRA): merge into the 16-bit weights, then quantize again.
- Or keep the adapters separate: one base model, a small file per task, swapped in.

QLoRA

LoRA on a 4-bit base.

Shrink the frozen part too



With LoRA, the frozen base still has to sit in memory: 14 GB in 16-bit for a 7B model - leaving almost nothing of a 16 GB GPU for activations and the adapters' Adam state. QLoRA stores the base in 4 bits (LLM-12): 3.5 GB of raw weights, about 12 GB to spare.

The adapters train in 16-bit; the base is turned back into 16-bit on the fly for each multiply.

A 7B model can be fine-tuned on a free Colab T4 with 16 GB. Dettmers et al. (2023) fine-tuned a 65B model on a single 48 GB GPU, matching 16-bit fine-tuning.

What QLoRA added

- **NF4**, a 4-bit format whose 16 levels sit at the quantiles of a normal distribution: each level covers the same share of weights, so the levels are dense where trained weights are dense (near 0). Blocks with a scale each, as in LLM-12.
- **Double quantization**: the block scales are themselves quantized - about 0.37 bits per parameter saved, some 3 GB on a 65B model.
- **Paged optimizers**: when GPU memory spikes, optimizer state is paged out to CPU RAM and back - slower for a moment, but no crash.

QLoRA's Guanaco models reached 99.3% of ChatGPT's level on the Vicuna benchmark (a judge-based test, LLM-11) - from a single GPU.

The recipe

What goes wrong, and how to check.

The recipe

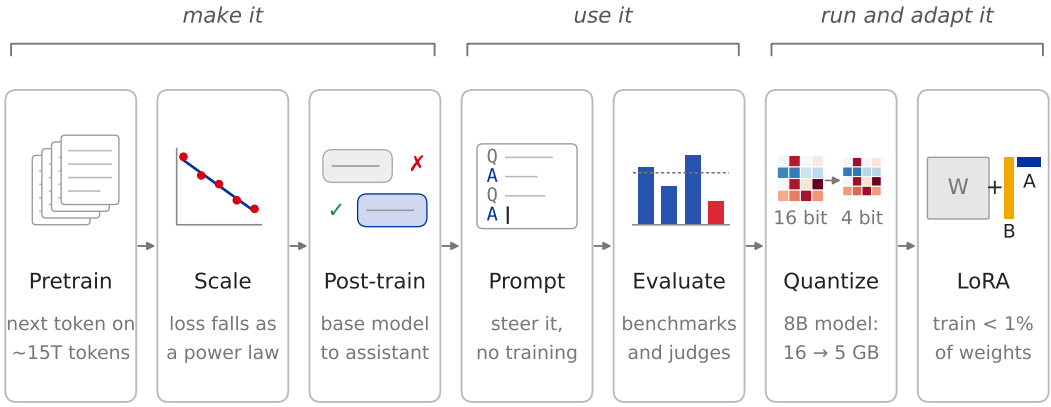
- **Data**: a few hundred to a few thousand clean examples beat many messy ones (LIMA, LLM-9). Ours: 32.
- **The chat template must match** (LLM-9): train and use the model in the same format, special tokens included.
- **Score only the answers**: the loss skips the prompt's tokens (our run did this).
- **Evaluate before and after** on held-out questions (LLM-11) - as the 20 unseen questions did here.
- **Watch for catastrophic forgetting**: a model fine-tuned hard on one task can get worse at others. Test a few general questions too.

Before converting a fine-tuned model for a laptop engine (LLM-12), merge the adapter into the weights: not every engine loads separate adapters.

Recap

- **Fine-tune last**: prompt, then documents, then fine-tune - for format and style, not facts.
- **LoRA**: freeze W, learn $\Delta W = BA$ of low rank; $B = 0$ so step 0 is the base model. 0.24% of GPT-2's weights taught it a new habit.
- **Merge** for zero extra cost at run time, or keep small adapters per task. Fine-tuning can cause forgetting: test general questions too.
- **QLoRA**: the frozen base in 4-bit NF4 - a 7B model fits a 16 GB GPU.
- **The recipe**: clean data, the right template, answers-only loss, evaluate before and after.

Next on the map



Next (LLM-14): what changed inside a 2026 LLM since GPT-2, and what each change costs.

Sources & further reading

- Hu et al. (2021), "LoRA: Low-Rank Adaptation of Large Language Models".
- Dettmers et al. (2023), "QLoRA: Efficient Finetuning of Quantized LLMs".
- Aghajanyan et al. (2020), "Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning".
- Zhou et al. (2023), "LIMA"; Gururangan et al. (2020), "Don't Stop Pretraining".

Figures: `py_src/lora_figs.py`. Real: the LoRA run on the cached GPT-2 (written by hand, one run, seed 509). By hand: the parameter counts and the memory bars.